

COMP517 – Data Analysis

Lab 4 –Explanatory Data Analysis: Univariate Visualisation

Data comes in various forms, requiring a range of distinct visualization methods to effectively facilitate understanding. Whether it's quantitative information like numerical values or qualitative data like categorical distinctions, adapting the visualization approach to match the characteristics of the data can significantly enhance its effectiveness. By employing appropriate visualization strategies, complex data sets can be transformed into insightful visual representations that not only simplify complexity but also uncover meaningful patterns and relationships hidden within the data. From bar graphs and histograms to pie charts and scatter plots, each visualization method provides a perspective for examining and interpreting data. This provides an accurate communication of underlying information while also enhancing accessibility and comprehension for a wider range of audiences.

The purpose of this lab is providing an overview of the various plots and charts that are commonly used in 'univariate' data visualisation.

Technical requirements:

- For the purpose of this lab, we will be working with a single CSV file named `sample_video_games_sales.csv`. Download the dataset and make sure to save the CSV file in the current directory where your `.ipynb` file is located. Alternatively, provide the accurate file path if it's stored in a different location. The dataset contains information regarding the sales of video games across various regions like North America, Europe, Japan and globally, while also giving information regarding the Names, Publishers, and Platforms. See Appendix A for more information about the dataset.
- Throughout the lab, we will be utilizing two essential Python libraries and modules: `pandas`, `NumPy`, `scipy.stats`, `seaborn` and `matplotlib`. Before proceeding, make sure that you have these libraries installed in your environment.
 - If you haven't installed it already, you can do so using `pip`.
 - Once you have the modules installed, import the libraries in your Python script using the import statement. Below is the list of modules/libraries that are needed in this lab:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats
```

The line `import matplotlib.pyplot as plt` is used to import the `pyplot` module from the `matplotlib` library and give it the alias `plt`.

By using the `plt` alias, provided when you imported the `pyplot` module, you can access the functions and methods provided by the `pyplot` module.

By using the `sns` alias, provided when you imported the `seaborn` library, you can create visually appealing and informative statistical graphics.

Univariate Data Visualisation:

Data Loading: Load the dataset and inspect the first few rows and data types of columns using the `df.head()` and `df.info()` methods.

```
1 # Read the cleaned CSV file
2 df = pd.read_csv("video_games_sales.csv")
3
4 # Display the first few rows of the DataFrame
5 print(df.head())
6
7 # Display basic information about the DataFrame
8 print(df.info())
9
```

The data types of columns provided by `df.info()` can guide your choice of visualization methods. Different types of visualizations are suitable for different types of data.

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 16291 entries, 0 to 16290
Data columns (total 11 columns):
#   Column          Non-Null Count  Dtype
---  ---
0   Rank            16291 non-null  int64
1   Name            16291 non-null  object
2   Platform        16291 non-null  object
3   Year            16291 non-null  float64
4   Genre           16291 non-null  object
5   Publisher       16291 non-null  object
6   NA_Sales        16291 non-null  float64
7   EU_Sales        16291 non-null  float64
8   JP_Sales        16291 non-null  float64
9   Other_Sales     16291 non-null  float64
10  Global_Sales    16291 non-null  float64
dtypes: float64(6), int64(1), object(4)
```

1. Visualization of Amounts:

When representing numerical values for specific categories, the most prevalent method is through bar graphs. These bars can be oriented either vertically or horizontally, and an alternative approach involves placing dots at the point where the corresponding bar would terminate.

➤ **Task1:** Create a visualization that displays the distribution of game genres in the dataset.

By using the **plt** alias, was provided when you imported the **pyplot** module, you can access the functions and methods provided by the **pyplot** module.

Step 1: Count the occurrences of each genre

```
genre_counts = df["Genre"].value_counts()
```

The above code uses pandas `value_count()` method to count the occurrences of each unique value in the "Genre" column and returns a new Series called `genre_counts` with the counts, indexed by the unique values.

Step 2: Extract genre names and their respective counts

```
genres = genre_counts.index
counts = genre_counts.values
```

The above lines, extract the index and values from the `genre_counts` pandas series that we created in Step 1.

Step 3: Set the figure size

```
plt.figure(figsize=(10, 6))
```

The `plt` alias calls the **figure()** function to create a new figure (plot canvas). The **figsize** parameter is used to specify the dimensions (width and height) of the figure in inches. In above code, the width of the figure is set to 10 inches and the height is set to 6 inches.

Step 4: Create the bar plot

```
plt.bar(genres, counts)
```

bar() function from the **pyplot** module creates a bar plot where the **genres** list (x-values) represents the categories (unique game genres) and the **counts** list (y-values) represents the heights of the bars (count of games for each genre).

Step 5: Rotate x-axis labels for better readability

```
plt.xticks(rotation=45)
```

You can rotate the x-axis labels in a bar plot to improve their readability. [See more about customization!](#)

Step 6: Add labels and title

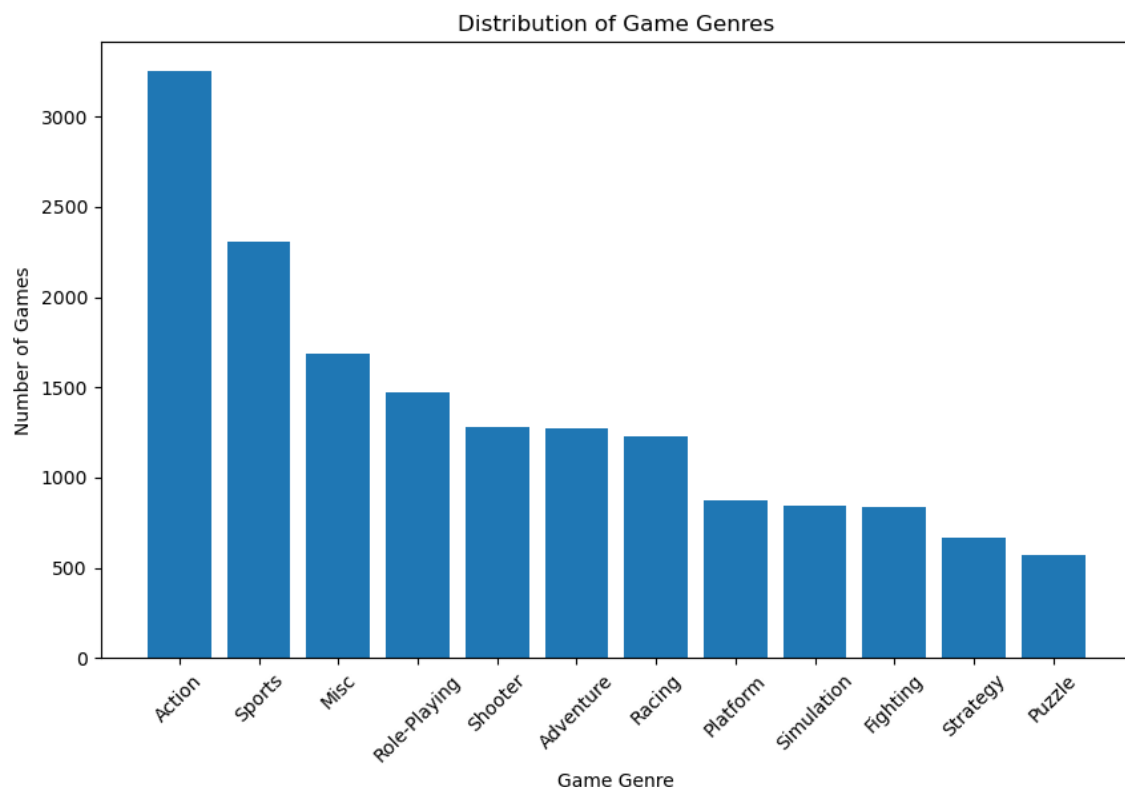
```
plt.xlabel("Game Genre")
plt.ylabel("Number of Games")
plt.title("Distribution of Game Genres")
```

- the **xlabel()** function from the **pyplot** module adds a label to the x-axis of the plot, describing what the x-axis represents.
- the **ylabel()** function from the **pyplot** module adds a label to the y-axis of the plot, describing what the y-axis represents.
- the **title()** function from the **pyplot** module adds a title to the plot, providing a brief description of what the plot is showing.

Step 7: Show the plot

```
plt.show()
```

the **show()** function from the **pyplot** module displays the plot that has been created using the preceding **plt.bar()** function or other plotting functions.



Q1: Based on the generated plot, explain your findings.

By using the **sns** alias, you can create visually appealing and informative statistical graphics.

Step 1: Set the figure size.

Step 2: Create a bar plot for the distribution of game genres.

```
# Create a bar plot for the distribution of game genres
sns.countplot(data=df, x="Genre", order=df['Genre'].value_counts().index)
```

By using the above code the **countplot()** function from **seaborn** is called.

- **countplot()** a count plot based on the data in the DataFrame **df** which the data will be taken for visualization is created,
- **x="Genre"**: the "Genre" column of the DataFrame is plotted on the x-axis.
- **order=df['Genre'].value_counts().index**: the order of the bars in the plot is set. The **value_counts()** method counts the occurrences of each genre in the "Genre" column and then extracts the index (genre names) in descending order of frequency. This ensures that the most common genres appear first in the plot.

Step 3: Rotate x-axis labels

Step 4: Add labels and title.

Step 5: Show the plot.

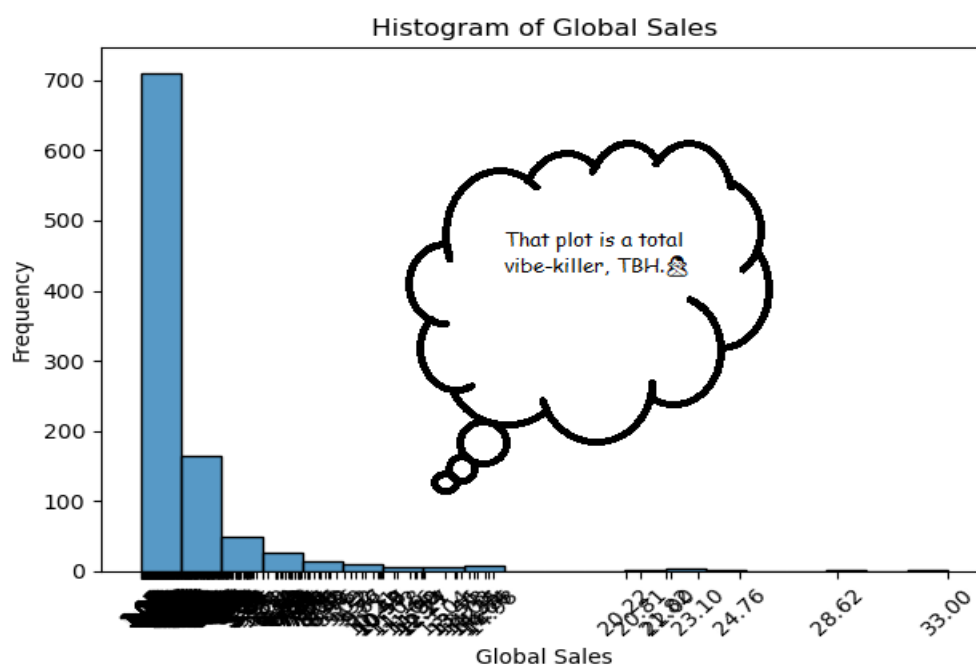
Provide the generated plot.

2. Visualizing Distribution:

To illustrate the distribution of data, histograms and boxplots stand out as the most intuitive options. These visualizations provide valuable insights into the spread and central tendencies of a dataset.

- **Task 2:** Generate a histogram to visualize the distribution of global sales values in the dataset. The x-axis should present the global sales values, and the y-axis represents the frequency of occurrences for each sales range.

```
# Create a histogram for Global Sales
sns.histplot(data=df, x="Global_Sales", bins=20)
plt.xlabel("Global Sales")
plt.ylabel("Frequency")
plt.title("Histogram of Global Sales")
# Use unique sales values as tick positions
plt.xticks(df["Global_Sales"].unique())
# Rotate x-axis labels for better readability
plt.xticks(rotation=45)
# Adjust layout and display the plot
plt.tight_layout()
plt.show()
```



We can customize the x-axis tick positions and labels on a plot and add a KDE plot to the visualization. In data visualization, Kernel Density Estimation (KDE) is a technique used to estimate the probability density function (PDF) of a continuous random variable based on a set of data points. KDE is particularly useful for visualizing the shape of a data distribution. When you set **kde=True**, seaborn adds a KDE plot to the visualization.

```
# Create a histogram for Global Sales
sns.histplot(data=df, x="Global_Sales", bins=20, kde=True,
label="Data Distribution")

plt.xlabel("Global Sales")
plt.ylabel("Frequency")
plt.title("Histogram of Global Sales")

# Add annotations to the x-axis for specific values
annotated_values = [1, 10, 20, 30]

plt.xticks(annotated_values, labels=[f"${value:.2f}M" for value in
annotated_values])

# Rotate x-axis labels for better readability
plt.xticks(rotation=90)

# Adjust layout and display the plot
plt.tight_layout()
plt.show()
```

Q2: Take a closer look at the plot you just generated and assess its skewness. Think about whether the distribution of data seems balanced or if it's leaning towards one side. Does it have a symmetrical bell shape, or does it show a longer tail on one side? Share your thoughts on whether the plot appears positively skewed (tail to the right), negatively skewed (tail to the left), or approximately symmetric.

Note: Skewness is just one aspect of a distribution's shape, and it provides insights into its asymmetry. The skewness value indicates the direction and degree of skew of the distribution:

- If skewness is negative, the distribution is skewed to the left (negatively skewed).
- If skewness is positive, the distribution is skewed to the right (positively skewed).
- If skewness is close to zero, the distribution is approximately symmetric.

To calculate the skewness of a dataset or a distribution, you can use the **scipy.stats.skew()** function from the SciPy library in Python. Skewness measures the asymmetry of the probability distribution of a real-valued random variable.

```

from scipy.stats import skew

# Calculate skewness of the "Global_Sales" column
global_sales_skew = skew(df["Global_Sales"])
print("Skewness of Global Sales:", global_sales_skew)

```

Q3: "What is the value of skewness for the Global Sales, and what does its value imply about the distribution's shape and asymmetry?"

Boxplots: Using a boxplot for data visualization offers several advantages and insights into the distribution of a dataset. That includes identifying central tendency, visualizing spread, detecting outliers, comparing distributions, handling skewed data presentation etc.

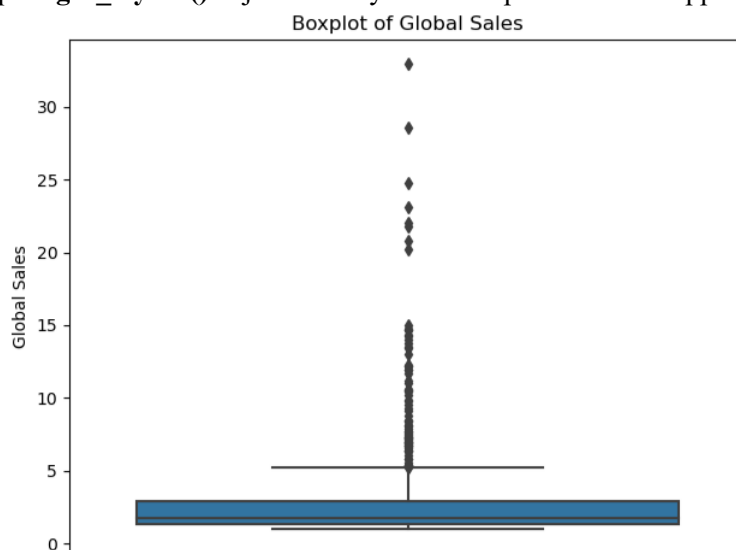
Task: generate a boxplot of the "Global_Sales" data using Seaborn and Matplotlib.

```

# Create a boxplot for Global Sales
sns.boxplot(data=df, y="Global_Sales")
plt.ylabel("Global Sales")
plt.title("Boxplot of Global Sales")
# Adjust layout and display the plot
plt.tight_layout()
plt.show()

```

The y-axis will represent the "Global_Sales" data, and the boxplot will be vertical. The **plt.tight_layout()** adjusts the layout of the plot for better appearance.



The whiskers of the boxplot extend from the edges of the blue box to the minimum and maximum data values that fall within a certain range. This range is often defined as 1.5 times the interquartile range (IQR), which is the range between the first quartile (Q1) and the third quartile (Q3). Data points that fall outside the whiskers are plotted individually as points on the plot. These points are considered outliers and are visually distinct from the rest of the boxplot.

You can customise the boxplot for the "Global_Sales" to include customizations for the appearance of markers for the minimum and maximum values, as well as the median and mean values:

```
# Set the figure size
plt.figure(figsize=(10, 6))

# Create a boxplot for Global Sales with means and a mean line
ax = sns.boxplot(data=df, x="Global_Sales", showmeans=True, meanline=False,
                 boxprops=dict(facecolor="lightgray"), meanprops=dict(markerfacecolor="blue", markeredgecolor="blue"),
                 medianprops=dict(color="red"))

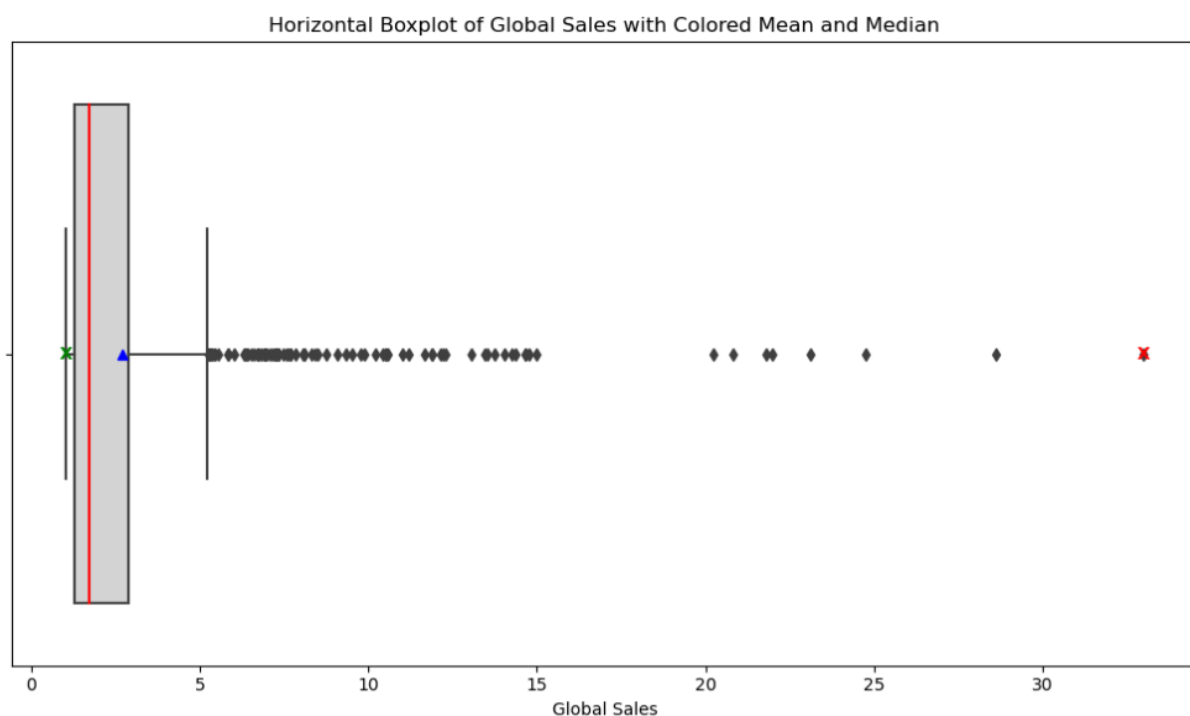
# Add annotations
median = df["Global_Sales"].median()
iqr = df["Global_Sales"].quantile(0.75) - df["Global_Sales"].quantile(0.25)
# Add markers for minimum and maximum values
min_value = df["Global_Sales"].min()
max_value = df["Global_Sales"].max()

plt.text(min_value, 0, "X", ha="center", va="center", color="green", fontweight="bold")
plt.text(max_value, 0, "X", ha="center", va="center", color="red", fontweight="bold")

plt.xlabel("Global Sales")
plt.title("Horizontal Boxplot of Global Sales with Colored Mean and Median")

# Display the plot
plt.tight_layout()
plt.show()
```

The generated output will look like below (I didn't say it is going to be pretty!):



Explanation: The green "X" marker shows the minimum value for Global_Sale. The red "X" marker shows the maximum value for Global_Sale. The red line represents the median line and the blue triangle

shows the mean value. The Global_Sale outliers have values that are much higher than most of the data, therefore they contributed significantly to the sum and subsequently affected the mean. On the other hand, the median is less sensitive to outliers as it is the middle value when the data is sorted. Therefore, it is not as influenced by extreme values as the mean. This property makes the median a more robust measure of central tendency in the presence of outliers.

Subplots: subplots provide a structured and organized way to present multiple visualizations, making it easier for viewers to interpret and compare different aspects of the data. To able to compare the histogram and boxplot of Global_sale you can use a subplots as below:

```
# Create subplots
fig, axes = plt.subplots(1, 2, figsize=(14, 6))

# Create a boxplot in the first subplot
sns.boxplot(data=df, x="Global_Sales", ax=axes[0])
axes[0].set_xlabel("Global Sales")
axes[0].set_title("Boxplot of Global Sales")

# Create a histogram with KDE in the second subplot
sns.histplot(data=df, x="Global_Sales", kde=True, ax=axes[1])
axes[1].set_xlabel("Global Sales")
axes[1].set_ylabel("Frequency / Density")
axes[1].set_title("Histogram with KDE of Global Sales")

# Adjust layout and display the subplots
plt.tight_layout()
plt.show()
```

- **Task 3:** Detect outliers in 'Global Sales' based on a Z-score threshold, then create a scatter plot to visualize both outliers and non-outliers separately.

```
numeric_cols = ["Global_Sales"]

# Calculate Z-scores
z_scores = np.abs((df[numeric_cols] - df[numeric_cols].mean()) /
df[numeric_cols].std())

# Set a Z-score threshold for outlier detection (e.g., 3)
z_score_threshold = 3

# Create a mask for outlier detection
outlier_mask = z_scores > z_score_threshold

# Scatterplot Visualization to identify outliers
for col in numeric_cols:

    # Scatter plot with different markers for outliers and non-
    outliers

    plt.figure(figsize=(8, 6))

    plt.scatter(df.index[~outlier_mask[col]],
df[col][~outlier_mask[col]], marker='o', label='Non-Outliers',
color='blue')

    plt.scatter(df.index[outlier_mask[col]],
df[col][outlier_mask[col]], marker='x', label='Outliers',
color='red')

    plt.xlabel('Index')

    plt.ylabel(col)

    plt.title(f'Scatter plot for {col} with Outliers')

    plt.legend()

    plt.show()
```

Explanation of the code:

The first 4 lines identifies outliers in Global_Sale column. The details for outlier detection using z_score is explained in Lab 03.

for col in numeric_cols: for loop is used to iterate through each column specified in the **numeric_cols** list. In this example, the **numeric_cols** list contains the names of the numeric column, Global_Sale, only. The purpose of using the **for** loop is to demonstrate how you can apply the same outlier detection and visualization steps to each numeric column specified in the **numeric_cols** list. This approach enables you to generate separate scatter plots for each numeric column. By iterating through the

columns in the list, you streamline the process and avoid duplicating the code for each individual column.

The below code creates a scatter plot for non-outliers using blue circle markers ('o').

```
plt.scatter(df.index[~outlier_mask[col]], df[col][~outlier_mask[col]], marker='o', label='Non-Outliers', color='blue')
```

- **df.index[~outlier_mask[col]]** retrieves the indices of non-outliers.
- **df[col][~outlier_mask[col]]** retrieves the corresponding values of the "Global_Sales" column for non-outliers.
- **marker='o'** specifies circle markers for non-outliers.
- **label='Non-Outliers'** sets the label for the legend.
- **color='blue'** sets the color of the markers to blue.

The below code creates a scatter plot for outliers using red 'x' markers.

```
plt.scatter(df.index[outlier_mask[col]], df[col][outlier_mask[col]], marker='x', label='Outliers', color='red')
```

- **df.index[outlier_mask[col]]** retrieves the indices of outliers.
- **df[col][outlier_mask[col]]** retrieves the corresponding values of the "Global_Sales" column for outliers.
- **marker='x'** specifies 'x' markers for outliers.
- **label='Outliers'** sets the label for the legend.
- **color='red'** sets the color of the markers to red.

The below codes are responsible for setting the labels of the x-axis and y-axis, respectively, in the scatter plot:

```
plt.xlabel('Index')
plt.ylabel(col)
```

- **plt.xlabel('Index')** sets the label for the x-axis of the scatter plot. "Index" represents the index values of the DataFrame.
- **plt.ylabel(col):** sets the label for the y-axis of the scatter plot. The **col** variable is the name of the specific numeric column that is currently being processed in the loop. In this example, **col** holds "Global_Sales," therefore y-axis label will be "Global_Sales."

Finally, set the title of the scatter plot and add a legend, respectively.

```
plt.title(f'Scatter plot for {col} with Outliers')
```

- The f-string format is used to dynamically insert the column name into the title. For example, if col is "Global_Sales," the title will be "Scatter plot for Global_Sales with Outliers."

```
plt.legend()
```

plt.legend() adds a legend to the scatter plot. The legend provides a visual guide to the different elements in the plot, such as markers for non-outliers and outliers. It uses the labels specified in the `plt.scatter()`, 'Non-Outliers' and 'Outliers', to distinguish between non-outliers and outliers.

Q4: Compare the outliers detected using Z-scores and those identified by the box plot. Do you notice any differences between the two methods? If there are differences, can you speculate on the possible causes of such discrepancies?

3. Depicting Proportions:

When dealing with proportions, several visualization techniques prove effective. Pie charts, side-by-side bars, and stacked bars are commonly employed. Pie charts are particularly adept at emphasizing the cumulative nature of individual components and highlighting simple fractions.

- **Task 4:** Generate a pie chart to visually represent the proportions of different game genres in the dataset.

```
# Calculate genre proportions
genre_counts = df["Genre"].value_counts()
genre_proportions = genre_counts / genre_counts.sum()

# Create a pie chart
plt.figure(figsize=(8, 8))

plt.pie(genre_proportions, labels=genre_proportions.index,
        autopct="%1.1f%%", startangle=140)

plt.title("Proportions of Game Genres")

# Equal aspect ratio ensures that pie is drawn as a circle.
plt.axis("equal")

# Display the pie chart
plt.show()
```

1. **genre_counts** keeps the count of each game genre in the dataset. It's calculated using the **value_counts()** method on the "Genre" column of the DataFrame.

2. **genre_counts.sum()** calculates the sum of all the genre counts, being the total number of games in the dataset.
3. **genre_proportions** takes the calculated proportions of each genre. By dividing each genre count by sum of counts, you get the proportion of each genre out of the total.
4. The **autopct="%1.1f%%"** parameter controls the format of the percentage values displayed on each slice of the pie chart with one decimal place and adds the percentage symbol.
5. The **startangle=140** parameter sets the starting angle for the first slice of the pie chart. By default, the starting angle is 0 degrees (i.e., the "top" of the pie chart). Setting it to 140 degrees rotates the starting position for better visualization.

Q5: Inspect the generated plot and explain your findings. Hint: Each slice of the pie corresponds to a genre, and its size is proportional to the genre's contribution to the total dataset. The percentage values displayed on each slice provide additional information about the exact proportions.

Similar to amount visualizations, bars can be arranged either vertically or horizontally.

- **Task 5:** Plot the distribution of video games across different gaming platforms to help viewers understand which platforms have the most and least number of games.

```
# Set the figure size
plt.figure(figsize=(10, 6))

# Create a bar plot with counts for platforms
sns.countplot(data=df, x="Platform",
order=df['Platform'].value_counts().index)

# Set labels and title
plt.xlabel("Platform")
plt.ylabel("Number of Games")
plt.title("Distribution of Games by Platform")

# Rotate x-axis labels for better visibility
plt.xticks(rotation=45)

# Display the plot
plt.tight_layout()
plt.show()
```

This visualization is particularly helpful for observing trends and patterns in the gaming industry based on the platforms where games are available.

Q6: Provide the plot and describe your findings.

Appendix A

This dataset has been generously provided by Kaggle, a platform known for hosting a wide array of datasets and competitions. The original dataset comprises a collection of video games that have achieved sales surpassing 100,000 copies. Each row corresponds to a specific game, while the columns encompass various attributes.

The dataset encompasses information about:

- Rank: The game's overall ranking.
- Name: The title of the game.
- Platform: The platform on which the game was released.
- Year: The year in which the game made its debut.
- Genre: The genre classification of the game.
- Publisher: The entity responsible for publishing the game.
- NA_Sales, EU_Sales, JP_Sales, Other_Sales, Global_Sales: Sales figures attributed to the game in North America, Europe, Japan, other regions, and globally, respectively (all measured in millions).

Note: For the purpose of this lab, the dataset has undergone modifications, including a reduction in size. As a result, the dataset now contains 1001 rows.